

Understanding Claude. Day 1

What is Claude?

Understanding LLM

Key terms to know

Transformer intuition

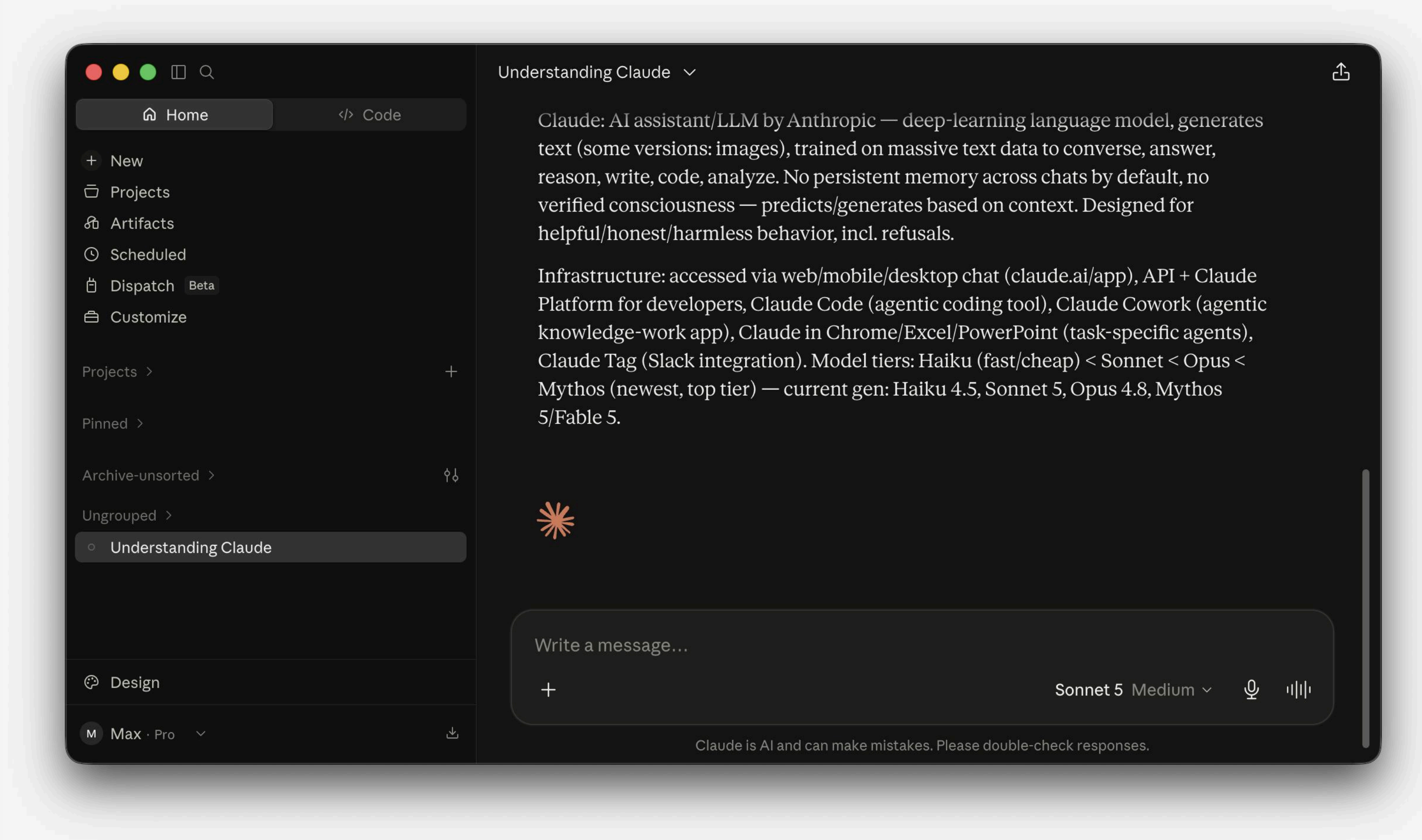
Capabilities and limitations

Why Claude?

What is Claude?

AI assistant/LLM by Anthropic – deep-learning language model, generates text (some versions: images), trained on massive text data to converse, answer, reason, write, code, analyze. No persistent memory across chats by default, no verified consciousness – predicts/generates based on context. Designed for helpful/honest/harmless behavior, incl. refusals.

Infrastructure: accessed via web/mobile/desktop chat, API, Claude Code, Claude Cowork, Claude in Chrome/Excel/PowerPoint.



How to access Claude

- **claude.ai / Claude app (web, mobile, desktop) – main chat interface**
- **API + Claude Platform – for developers building apps**
- **Claude Code – agentic coding tool (terminal, desktop, or mobile)**
- **Claude Cowork – agentic knowledge-work app for non-devs (desktop, remote via mobile)**
- **Claude in Chrome – browsing agent**
- **Claude in Excel – spreadsheet agent**
- **Claude in PowerPoint – slides agent**
- **Claude Tag – Slack integration, tag @Claude to delegate tasks**

Understanding LLM

At its core, a large language model (LLM) is a type of deep learning system trained on massive datasets of text to understand and generate human language. The fundamental task is deceptively simple: given a sequence of words, predict what comes next.

What makes a language model "large" is its parameter count: the adjustable weights that the model learns during training. Modern LLMs contain billions to trillions of parameters.

Understanding LLM

An LLM predicts language; it does not “know”

- A large language model is trained on vast amounts of text.
- It learns statistical patterns between pieces of language.
- Given some text, it predicts what is likely to come next.
- Its impressive capabilities emerge from **repeating this simple task at enormous scale.**

Understanding LLM

Tokens are the pieces an LLM reads and writes

- LLMs process **tokens**, not whole sentences or ideas.
- A token may be a word, part of a word, punctuation, or a symbol.
- Example: "Unbelievable!" might become Un + believ + able + !
- Both the prompt and the response consume tokens.
- **The context window** limits how many tokens the model can consider at once.

Understanding LLM

A response is generated one token at a time

- The model receives the tokens written so far.
- It assigns probabilities to possible next tokens.
- One token is selected.
- The new token is added to the sequence.
- The process repeats until the response ends.

Understanding LLM

The model is only one part of the application

The model provides language prediction, learned patterns and general capabilities in a limited context window.

The application adds system instructions and user interface, conversation history or memory, search, databases, calculators, and other tools, safety rules, permissions, and output formatting.

Understanding LLM

Fluent output is not guaranteed truth

- False statements can have the same confident tone as correct ones.
- Missing context or outdated knowledge increase error risk.
- Models “invent” details when the available evidence is insufficient.
- Important claims should be checked against reliable sources or grounded through tools.

Understanding LLM: a brief history

The beginning

From 1906 to 1912, Ferdinand de Saussure taught Indo-European linguistics, general linguistics, and Sanskrit at the University of Geneva. During this time he developed the foundation for a highly functional model of languages as systems. His book **Language as a Science**, published in 1916, evolved into natural language processing (NLP) – the theory behind LLMs.

Understanding LLM: a brief history

ELIZA

In 1966, an MIT computer scientist, Joseph Weizenbaum, developed ELIZA, which is described as the first program using NLP. It could identify keywords from the input it received, and respond with a pre-programmed answer. The author was surprised (perhaps even horrified) that people, including Weizenbaum's own secretary, described the computer program as having human-like feelings.

Understanding LLM: a brief history

Small Language Models

Early development of the first (small) language models was started in the 1980s by IBM, and they were/are designed to predict the next word in a sentence. Part of their design includes a “dictionary,” which determines how often certain words occur within the text the model was trained on. After each word, the algorithm recalculates statistically what the following word should be.

Understanding LLM: a brief history

WWW: a massive source of data

Tim Berners-Lee thought of the World Wide Web (WWW) in 1989, and it was made available to the public in 1991.

The creation of the World Wide Web made the internet searchable and provided large language models with access to massive amounts of information. The World Wide Web offers a platform to create, store, locate, and share information on a variety of topics.

Understanding LLM: a brief history

GPUs and Large Language Models

Large language models require complex training, which includes the use of huge amounts of data containing billions of words and phrases. Training large language models can be described as training the individual pieces of a massive jigsaw puzzle, with each puzzle piece representing a portion of the LLM's understanding. GPUs (graphics processing units) provide a solution to these problems.

Understanding LLM: a brief history

Deep Learning

In the 1990s, the arrival of deep learning supported even more advanced language models. A large language model is a very large deep learning model that is pre-trained on massive amounts of data. Deep learning is a form of machine learning, which is also a neural network, but with additional layers. In 2011, deep learning began becoming popular. By 2018, deep learning algorithms were being used in every industry, from photography to online detail.

Understanding LLM: a brief history

- 11 Jun 2018 OpenAI GPT
- 30 Nov 2022 OpenAI ChatGPT
- 24 Feb 2023 Meta LLaMA
- 14 Mar 2023 Anthropic Claude
- 3 Nov 2023 xAI Grok
- 29 Nov 2023 DeepSeek DeepSeek LLM
- 6 Dec 2023 Google Gemini

Key terms to know

- **Model** – learned system mapping inputs to outputs.
- **Parameter** – learned numerical weight inside model.
- **Token** – basic text unit; word, subword, punctuation.
- **Tokenizer** – converts text $\leftrightarrow$ tokens.
- **Context window** – maximum tokens model can process at once.
- **Prompt** – instructions/context given to model.
- **Pretraining** – broad training on large datasets.

Key terms to know

- **Post-training** — alignment and capability refinement after pretraining.
- **Fine-tuning** — additional training for specific behavior/domain.
- **RLHF** — training from human preference feedback.
- **Probability distribution** — likelihood assigned to each possible next token.
- **Temperature** — randomness control; lower = more predictable.
- **Max output tokens** — generation-length limit.

Key terms to know

- **Transformer** — dominant LLM architecture.
- **Attention** — mechanism relating tokens to relevant tokens.
- **Self-attention** — attention among tokens in the same sequence.
- **Embedding** — vector representation of text or other data.
- **Layer** — repeated processing block in a neural network.
- **Multimodal model** — processes text plus images, audio, or video.
- **Hallucination** — plausible but unsupported or false output.

Key terms to know

- **RAG (Retrieval-Augmented Generation)** — retrieve documents, then generate using them.
- **Chunking** — splitting documents into retrievable pieces.
- **Tool calling** — model requests external functions or APIs.
- **Agent** — model-driven system that plans, uses tools, and acts.
- **Memory** — retained information across interactions.

Key terms to know

- **Guardrail** — rule or mechanism limiting unsafe/invalid behavior.
- **Latency** — time required to produce a response.
- **Quantization** — lower-precision weights for faster, cheaper inference.
- **Distillation** — training a smaller model from a stronger model.
- **Open-weight model** — downloadable model weights; not necessarily open-source.

Transformer intuition

Transformer: code idea

- Repeated information routing + transformation
- Input $\rightarrow$ tokens $\rightarrow$ vectors
- Residual stream carries accumulated information
- Many layers $\rightarrow$ richer contextual representations

Transformer intuition

Tokens as contextual representations

A token starts as a vector containing learnable features. Its representation changes layer by layer.

For example, **bank** in **river bank** and **central bank** starts similarly → becomes increasingly different.

Transformer intuition

Self-attention

Each token effectively asks:

- Query: What am I looking for?
- Key: What do other tokens contain?
- Value: What information should I retrieve?

For example: The animal didn't cross the street because **it** was tired.

Transformer intuition

Multi-head attention

Transformers use many attention heads in parallel. Different heads may track:

- syntax
- entities
- previous tokens, repeated patterns
- long-range dependencies

Transformer intuition

MLP = Multi-Layer Perceptron

A small feed-forward neural network applied independently to each token position.

Attention answers: Where should I get information from?

MLP answers: What should I do with the information I now have?

MLPs can detect and create useful features.

Transformer intuition

Layers build complex behavior

One layer performs relatively simple operations. Many layers compose them: **retrieve → transform → retrieve → combine → transform**

Rough intuition:

- early: local patterns, syntax
- middle: entities, structure, semantics
- later: combine evidence, prepare predictions

Transformer intuition

What the Transformer outputs

The model repeatedly predicts: What token should come next?

This enables: in-context learning, pattern completion, and flexible task adaptation.

Transformer is a learned system for routing and transforming information to predict continuation.

Capabilities and limitations: strength

- Summarization and rewriting
- Information extraction
- Classification and tagging
- Code generation and explanation
- Comparing alternatives
- Following examples and output formats
- Adapting explanations to context

Capabilities: tool-augmented

- With external data access (tool-augmented), LLM systems can:
 - Search current information
 - Retrieve internal documents
 - Read large files beyond prompt limits
 - Look up specific facts instead of guessing
 - Combine information from multiple sources
- Retrieval reduces dependence on training data.

Capabilities: actions

- With tools, LLM systems can:
 - Execute code
 - Run calculations
 - Query databases and APIs
 - Analyze images and documents
 - Create or modify files
 - Trigger external actions

Capabilities: best task types

Best results when:

- Task is clearly defined
- Relevant context is available
- Output can be verified
- Work can be split into steps
- Examples show the expected result

generation • transformation • extraction • analysis • coding assistance

Limitations: knowledge

- Training knowledge may be outdated
- Missing information may be invented
- Sources can be fabricated
- Similar concepts may be confused
- Rare or highly specialized facts are less reliable
- Fluent wording can hide uncertainty

Limitations: reasoning and context

- Can miss constraints
- Can make logical or arithmetic errors
- Early wrong assumption can affect the whole answer
- Long context can reduce attention to important details
- Ambiguous instructions produce unstable results
- Same prompt can produce different answers

Capabilities and limitations: summary

LLMs are strongest as reasoning + generation interfaces, not reliable sources of truth or a full-functioning worker.

Best pattern: LLM + context + tools + verification

Why Claude?

- **Frontier reasoning**
- **Massive context**
- **Agentic execution**

Current model lineup: Fable 5, Opus 5, Sonnet 5, Haiku 4.5

Why Claude?

- **Adaptive thinking.** Scales effort to task complexity, on by default.
- **Multimodal input.** Text, code, images, PDFs – one thread, one model.
- **1M-Token Context.** Enough for a full codebase or 750K-word doc.
- **Right-Sized Models.** Haiku → Sonnet → Opus → Fable, pick cost vs. capability.

Why Claude?

Rich ecosystem

- **Claude Code.** Agentic coding in a terminal, IDE, or the Claude app.
- **Artifacts & Cowork.** Docs and more built live, right in the chat.
- **MCP Connectors.** Open standard – plugs into Drive, Slack, GitHub, and more.
- **API.** Full model access, prompt caching, batches, tool use.